

Análisis de datos con R

Miguel García Moreno

2025-11-02

1 Abstract

En este proyecto se utiliza el lenguaje de programación R para realizar análisis de datos descriptivo y preprocesamiento de datos principalmente. Aun así en la última parte se construyen ajustes lineales y se procede con la selección y evaluación de modelos. Se trabaja principalmente con los fundamentos estadísticos básicos a través de R. El proyecto se realiza como parte de la asignatura FESTAD del máster de Data Science y Big Data de la Universidad de Sevilla.

Contents

1 Abstract	2
2 Instalación de paquetes	5
3 Ejercicio 1. Obtención de datos y preprocesamiento	6
3.1 ¿Cuántos coches (filas) tiene este dataset?	7
3.2 ¿Cuáles y de qué tipo son las variables de este dataset?	7
3.3 Obtenga un dataset de forma aleatoria que contenga el 75% de las filas de “dtcar”, llámelo: “dtcar.entre”.	7
3.4 ¿Cuántas observaciones NA hay en “dtcar.entre” en total?	8
3.5 Obtenga las frecuencias con las que aparecen las distintas modalidades de la variable “Puertas”, con ayuda de la función summary().	8
3.6 Convierte la variable “Puertas” en una nueva variable (fuera del dataset) “PuertasNum” que sea de tipo “numeric”, que contenga como valor numérico el número de puertas y que asigne a la modalidad “5more” el valor 5.	8
3.7 Calcule el número medio de la nueva variable “PuertasNum”.	8
3.8 Explique qué es lo que ocurre si se intenta ejecutar el siguiente código: dtcar\$Asientos[1:5] = 3	9
4 Ejercicio 2. Funciones R	9
5 Ejercicio 3. Preprocesamiento de datos y funciones R	11
6 Ejercicio 4. Tydiverse, dplyr	13
6.1 ¿Cuál fue el vendedor que hizo más ventas en total?	13
6.2 ¿En qué 3 categorías de producto se vendió más?	13
6.3 ¿Qué 5 clientes compraron más en el mes de enero de 2020?	14
6.4 ¿Qué 10 días del 2020 se vendió más? Mostrarlos de forma descendente, indicando la venta realizada y también el mes al que pertenecían.	14
6.5 Responda a la cuestión “3” sin usar el sistema tidyverse, es decir, usando recursos del sistema base.	15
6.6 ¿Qué 3 categorías de producto son las más vendidas?	15
6.7 ¿Qué producto ha comprado más el cliente que más ha comprado?	15

7	Ejercicio 5. Análisis y visualización de datos agrupados.	16
7.1	Instrucciones.	16
7.2	Conjuntos de datos.	16
7.3	Cargue los datos en R en un dataset. Muestre los 7 primeros casos de los datos. Guarde el dataset en un fichero RData: “Datos_tema2.RData”.	16
7.4	Análisis exploratorio de los datos	17
7.4.1	Calcule la tabla de frecuencias (absolutas, relativas y acumuladas) las variables glucosa e IMC, agrupándolas previamente en intervalos. Las tablas deben tener un máximo de 8 intervalos.	17
7.4.2	Obtenga una tabla formateada para latex con ayuda del paquete knitr.	18
7.4.3	Obtenga una tabla formateada para html con ayuda del paquete kableExtra.	18
7.4.4	Calcule las medidas estadísticas vistas para las variables Presión arterial y Función DiabetesPedigree. Aplique también las funciones de resumen descriptivo, así como el cálculo de las medidas de concentración.	18
7.4.5	Estudie e identifique los posibles outliers de la variable Presión arterial. Acompañe con las gráficas adecuadas.	20
7.4.6	Obtenga el histograma de al menos la variable Insulina.	21
7.4.7	Seleccione la variable Glucosa y agrupe sus valores en cuatro intervalos.	22
7.4.8	En cada uno de los grupos definidos por los grupos de esa segunda variable, obtenga el histograma de la primera variable y represente los histogramas conjuntamente. . . .	23
7.4.9	Obtenga la tabla de frecuencias conjuntas de las variables usadas en el apartado anterior (ya agrupadas en intervalos)	24
7.4.10	Represente las frecuencias conjuntas mediante una representación gráfica adecuada. . .	24
7.4.11	Seleccione las variables Edad y Espesor de la piel. Agrupar la primera de ellas en 5 intervalos e identifique los posibles outliers de la segunda variable en cada grupo definido por los intervalos de la primera variable. Usando la librería ggplot2, acompañe con las gráficas adecuadas.	25
8	Ejercicio 6. Ajuste lineal	27
8.1	Genere 200 valores de una distribución $N(0,1)$ tomando como semilla 1125. Guardelos resultados en un vector X.	27
8.2	Genere 200 valores de una distribución normal de media cero y varianza 0.36 . Guarde los resultados en un vector ϵ	28
8.3	Utilizando los vectores X y epsilon, obtenga un vector Y descrito por el modelo $Y = -1 + 0.5X + \epsilon$ ¿Qué longitud tiene el vector Y? ¿Cuáles son los valores de β_0 y β_1 en este modelo? .	29
8.4	Considere los datos, $\{(x_i, y_i)\}$, realice un gráfico para mostrar la relación entre estos valores. .	29
8.5	Ajuste un modelo de regresión lineal	30
8.6	Represente el modelo	30
8.7	Comente los resultados obtenidos (compare $\hat{\beta}_0$, $\hat{\beta}_1$ y $\hat{\sigma}^2$ con β_0 , β_1 y σ^2).	31

9	Ejercicio 7. Ajuste lineal por subconjuntos. Selección de modelo. Evaluación de modelos.	31
9.1	Obtenga el mejor modelo con 3 variables predictoras utilizando regsubsets. Lo denotaremos por M3. Incluya y comente alguno de los gráficos que pueden obtenerse con regsubsets. . . .	32
9.2	Ecuación del modelo M3, interpretación de los coeficientes. Tests y medidas de bondad de ajuste de este modelo.	34
9.3	En M3, estimación del valor medio de nox.	35
9.4	Halle el intervalo de confianza y la predicción para este valor para los valores (70, 4, 310).Muestre los resultados en una tabla.	35
9.5	Justifica la validez de la predicción.	36
9.6	Métodos de selección de variables paso a paso. Aplica el método de selección de variables "both". Comenta los resultados obtenidos, y escribe el modelo final, Mf , que propondrías. . .	36
9.7	Estudie la posible existencia de colinealidad en los predictores propuestos en el modelo Mf . .	40
9.8	Comparación de las medidas de bondad de ajuste de M3 y Mf : R^2 , R^2 adj , error standard residual y Raíz del Error Cuadrático Medio de los residuos (RCME). Realice una tabla en que se comparen las medidas de bondad de ajuste de estos modelos. Comente los resultados obtenidos.	40

2 Instalación de paquetes

```
suppressWarnings({
  {
paquetes<-c('kableExtra','writexl','tidyverse','dplyr','tibble','DistributionUtils','reldist','ggplot2',
            'leaps','MASS','car','patchwork')
for (i in 1:length(paquetes))
{
  if(require(paquetes[i],character.only = TRUE)) cat(paquetes[i], " se ha cargado correctamente\n")
  else {
    cat("Tratando de instalar",paquetes[i])
    install.packages(paquetes[i])
    if(require(paquetes[i],character.only = TRUE)) cat(paquetes[i], " correctamente cargado")
    else {
      stop("No ha podido instalarse", paquetes[i])
    }
  }
}
}
})
```

```
## Cargando paquete requerido: kableExtra
```

```
## Cargando paquete requerido: writexl
```

```
## Cargando paquete requerido: tidyverse
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
```

```
## v dplyr      1.1.4      v readr      2.1.5
```

```
## v forcats    1.0.1      v stringr    1.5.2
```

```
## v ggplot2    4.0.0      v tibble     3.3.0
```

```
## v lubridate  1.9.4      v tidyr      1.3.1
```

```
## v purrr      1.2.0
```

```
## -- Conflicts ----- tidyverse_conflicts() --
```

```
## x dplyr::filter()      masks stats::filter()
```

```
## x dplyr::group_rows() masks kableExtra::group_rows()
```

```
## x dplyr::lag()         masks stats::lag()
```

```
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
## Cargando paquete requerido: DistributionUtils
```

```
##
```

```
## Cargando paquete requerido: reldist
```

```
##
```

```
## reldist: Relative Distribution Methods
```

```
## Version 1.7-2 created on 2023-02-16.
```

```
## copyright (c) 2003, Mark S. Handcock, University of California-Los Angeles
```

```
## For citation information, type citation("reldist").
```

```
## Type help(package="reldist") to get started.
```

```
##
```

```
##
```

```
## Cargando paquete requerido: ISLR2
```

```
##
```

```
## Cargando paquete requerido: leaps
##
## Cargando paquete requerido: MASS
##
##
## Adjuntando el paquete: 'MASS'
##
##
## The following object is masked from 'package:ISLR2':
##
##     Boston
##
##
## The following object is masked from 'package:dplyr':
##
##     select
##
##
## Cargando paquete requerido: car
##
## Cargando paquete requerido: carData
##
##
## Adjuntando el paquete: 'car'
##
##
## The following object is masked from 'package:dplyr':
##
##     recode
##
##
## The following object is masked from 'package:purrr':
##
##     some
##
##
## Cargando paquete requerido: patchwork
##
##
## Adjuntando el paquete: 'patchwork'
##
##
## The following object is masked from 'package:MASS':
##
##     area
```

3 Ejercicio 1. Obtención de datos y preprocesamiento

Elegimos la semilla.

```
set.seed(123)
```

Importe en R el dataset “dtcar” que se encuentra en la siguiente url: “<https://destio2008.us.es/calvo/>”

descargas/dtcar.RData”. Utilice R para responder a las siguientes cuestiones:

```
load("dtcar.Rdata")
```

3.1 ¿Cuántos coches (filas) tiene este dataset?

```
nrow(dtcar)
```

```
## [1] 1728
```

```
dtcar_len<-nrow(dtcar)
```

Hay 1728 filas.

3.2 ¿Cuáles y de qué tipo son las variables de este dataset?

```
str(dtcar)
```

```
## 'data.frame': 1728 obs. of 7 variables:
## $ Precio : Factor w/ 4 levels "high","low","med",...: 4 4 4 4 4 4 4 4 4 4 ...
## $ Mantenimiento: Factor w/ 4 levels "high","low","med",...: 4 4 4 4 4 4 4 4 4 4 ...
## $ Puertas : Factor w/ 4 levels "2","3","4","5more": 1 1 1 1 1 1 1 1 1 1 ...
## $ Asientos : Factor w/ 3 levels "2","4","more": 1 1 1 1 1 1 1 1 1 2 ...
## $ Maletero : Factor w/ 3 levels "big","med","small": 3 3 3 2 2 2 1 1 1 3 ...
## $ Seguridad : Factor w/ 3 levels "high","low","med": 2 3 1 2 3 1 2 3 1 2 ...
## $ Clase : Factor w/ 2 levels "acceptable","inacceptable": 2 2 2 2 2 2 2 2 2 2 ...
```

Las variables de este dataset son Precio, Mantenimiento, Puertas, Asientos, Maletero, Seguridad y Clase, todas ellas variables categóricas.

3.3 Obtenga un dataset de forma aleatoria que contenga el 75% de las filas de “dtcar”, llámelo: “dtcar.entre”.

```
sample_indexes<-sample(1:dtcar_len,size = dtcar_len*.75, replace = FALSE)
dtcar.entre<-dtcar[sample_indexes,]
head(dtcar.entre)
```

```
## Precio Mantenimiento Puertas Asientos Maletero Seguridad Clase
## 415 <NA> low 5more 4 small low inacceptable
## 463 high <NA> 3 2 med low inacceptable
## 179 vhigh high 4 4 <NA> med inacceptable
## 526 high vhigh 5more 4 med low <NA>
## 195 vhigh high 5more 2 <NA> high inacceptable
## 938 med vhigh 4 more <NA> med inacceptable
```

3.4 ¿Cuántas observaciones NA hay en “dtcar.entre” en total?

```
sum(is.na(dtcar.entre))
```

```
## [1] 10
```

Hay 10 observaciones NA.

3.5 Obtenga las frecuencias con las que aparecen las distintas modalidades de la variable “Puertas”, con ayuda de la función summary().

```
summary(dtcar.entre)
```

```
##      Precio      Mantenimiento  Puertas  Asientos  Maletero  Seguridad
## high :335    high :318      2    :317  2    :435  big   :431  high:434
## low  :320    low  :327      3    :324  4    :426  med   :436  low :432
## med  :323    med  :332      4    :333  more:435  small:426  med :429
## vhigh:314    vhigh:318    5more:322                NA's : 3    NA's: 1
## NA's : 4    NA's : 1
##           Clase
## acceptable :394
## inacceptable:901
## NA's       : 1
##
##
```

Las distintas frecuencias aparecen en la salida de la función summary() arriba.

3.6 Convierte la variable “Puertas” en una nueva variable (fuera del dataset) “PuertasNum” que sea de tipo “numeric”, que contenga como valor numérico el número de puertas y que asigne a la modalidad “5more” el valor 5.

```
levels(dtcar$Puertas)
```

```
## [1] "2"      "3"      "4"      "5more"
```

```
PuertasNum<-as.numeric(dtcar$Puertas)+1
head(PuertasNum)
```

```
## [1] 2 2 2 2 2 2
```

3.7 Calcule el número medio de la nueva variable “PuertasNum”.


```
mean(PuertasNum)
```

```
## [1] 3.5
```

3.8 Explique qué es lo que ocurre si se intenta ejecutar el siguiente código:
dtcar\$Asientos[1:5] = 3

```
dtcar$Asientos[1:5] = 3
```

```
## Warning in `[<-.factor`(`*tmp*`, 1:5, value = structure(c(NA, NA, NA, NA, :  
## invalid factor level, NA generated
```

La instrucción level muestra los las categorías del factor puertas. Por defecto asigna números naturales empezando por uno y de forma incremental a cada categoría. Por tanto al sumarle uno aseguramos que el valor numérico tenga una correspondencia con literal del texto. La media arimética de PuertasNum es 3.5. Y al intentar asignar un número a un factor R falla e introduce un valor NA en su lugar.

4 Ejercicio 2. Funciones R

Cree una función en lenguaje R, llamada: “t.central”, con las siguientes características:

El resultado que devuelve la función dependerá del tipo de vector que se le pase como primer argumento: vdatos. Si es:

- “character” o “factor”: devuelva valor/es más frecuente/s (use: table())
- “numeric”: devuelva la media.
- “integer”: devuelva la mediana.

Tenga un segundo argumento, na.rm, que permita eliminar los valores NA que pueda tener o no: vdatos.

```
set.seed(123)
```

```
t.central <- function(vdatos, na.rm = FALSE) {  
  if (na.rm == TRUE){  
    vdatos<-na.omit(vdatos)  
  }  
  
  vdatos_class=class(vdatos)  
  if (vdatos_class %in% c("character","factor")){  
    resultado = table(vdatos)  
  }  
  else if (vdatos_class == "numeric"){  
    resultado = mean(vdatos)  
  }  
  else if(vdatos_class == "integer"){  
    resultado = median(vdatos)  
  }  
}
```

```

else {
  stop("Data type not allowed!")
}

return(resultado)
}

```

Utilice la función “t.central” en las siguientes situaciones:

- Sobre el vector “v1” creado del siguiente modo: `v1 = sample(letters,100,TRUE)`
- Sobre el vector “v2” `v2 = as.integer(trunc(runif(100,0,50)))`
- Sobre el vector “v3”: `v3 = rnorm(100)`
- Sobre el data.frame “df01” cuyas columnas sean los vectores: v1, v2 y v3. Utilice la función R: `lapply()`

```

v1 = sample(letters,100,TRUE)
v2 = as.integer(trunc(runif(100,0,50)))
v3 = rnorm(100)
df01 <- data.frame(
  v1 = v1,
  v2 = v2,
  v3 = v3
)
v4=c(2,3,NA)

t.central(v1)

```

```

## vdatos
## a b c d e f g h i j k l m n o p q r s t u v w y z
## 1 2 5 2 4 3 6 4 5 4 3 3 3 6 4 3 3 3 4 3 4 8 3 10 4

```

```
t.central(v2)
```

```
## [1] 24
```

```
t.central(v3)
```

```
## [1] -0.01390126
```

```
t.central(v4,TRUE)
```

```
## [1] 2.5
```

```
lapply(df01,t.central)
```

```

## $v1
## vdatos
## a b c d e f g h i j k l m n o p q r s t u v w y z
## 1 2 5 2 4 3 6 4 5 4 3 3 3 6 4 3 3 3 4 3 4 8 3 10 4
##
## $v2
## [1] 24
##
## $v3
## [1] -0.01390126

```

5 Ejercicio 3. Preprocesamiento de datos y funciones R

Introduzca los siguientes datos:

- Alumno 1 (idalu = "id1"): 3, 6, 9, 8.5
- Alumno 2 (idalu = "id2"): 6, 7.5, 5, 3.5, 8
- Alumno 3 (idalu = "id3"): 4, 6, 7, 8, 7, 9
- Alumno 4 (idalu = "id4"): 2, 3, 5, 4, 7

Que corresponden a las calificaciones de 4 alumnos en la asignatura “matemáticas”, en un objeto de tipo “data.frame” (llámelo: “dtcalif”) que contenga únicamente dos columnas: “idalu” y “calificacion”.

```
set.seed(123)

idalu <- c(rep("id1",4), rep("id2",5), rep("id3",6),rep("id4",5))
calificacion <- c( 3, 6, 9, 8.5, 6, 7.5, 5, 3.5, 8, 4, 6, 7, 8, 7, 9, 2, 3, 5, 4, 7)

dtcalif <- data.frame(idalu, calificacion)
```

Obtenga la media y cuasidesviación típica de cada alumno, utilizando el objeto “dtcalif”.

```
print("Medias")
```

```
## [1] "Medias"
```

```
tapply(dtcalif$calificacion, dtcalif$idalu, mean)
```

```
##      id1      id2      id3      id4
## 6.625000 6.000000 6.833333 4.200000
```

```
print("Cuasidesviaciones")
```

```
## [1] "Cuasidesviaciones"
```

```
tapply(dtcalif$calificacion, dtcalif$idalu, sd)
```

```
##      id1      id2      id3      id4
## 2.750000 1.837117 1.722401 1.923538
```

Cree una función R que genere un objeto de tipo “list”, para el que cada uno de sus elementos contenga las distintas calificaciones de un alumno (generadas estas aleatoriamente entre 1 y 10) y el nombre del elemento sea el identificador del alumno: id1, id2, ... La función debe tener 3 argumentos:

- “num.alumnos”: número de alumnos que se generarán.
- “num.notas.min = 3”: número de notas que como mínimo puede tener asignadas un alumno.
- “num.notas.max = 10”: número de notas que como máximo puede tener asignadas un alumno.

```

generador_clases<-function(num.alumnos, num.notas.min = 3, num.notas.max = 10){
  clase = vector("list", length = num.alumnos)
  notas <- seq(from = 1, to = 10, by = 0.5)
  n_notas_posibles <- seq(from = num.notas.min, to = num.notas.max, by = 0.5)

  clase <- lapply(1:num.alumnos, function(i) {
    n_notas <- sample(n_notas_posibles, 1)
    sample(notas, n_notas, replace = TRUE)
  })

  names(clase) <- paste0("id", 1:num.alumnos)

  return(clase)
}

```

Genere 3 objetos de tipo “list” con ayuda de la función creada en el apartado anterior: “lclase01” (con 100 alumnos), “lclase02” (con 1000 alumnos), “lclase03” (con 1 millón de alumnos)

```

lclase01 = generador_clases(100)
lclase02 = generador_clases(1000)
lclase03 = generador_clases(1000000)

```

Guarde los tres objetos en un fichero RData llamado: “clases.RData”.

```

save(list = c("lclase01","lclase02","lclase03"), file = "clases.RData")

```

Guarde: “lclase01” en un fichero con formato “csv2”, “lclase02” en un fichero “xlsx” y “lclase03” en un fichero “rds”.

```

list_to_df <- function(class_list){
  df <- stack(class_list)
  names(df) <- c("notas", "estudiante")
  return(df)
}

write.csv2(list_to_df(lclase01), "lclase01.csv", row.names = FALSE)

write_xlsx(list_to_df(lclase02), "lclase02.xlsx")

df_lclase03 = list_to_df(lclase03)
saveRDS(df_lclase03, "lclase03.rds")

```

Obtenga la media y cuasidesviación típica de cada alumno en el objeto: lclase03 y guarde los resultados en un objeto llamado: “calificacionesalumnos.RData”.

```

print("Medias")

```

```

## [1] "Medias"

```

```
lclase03_mean = tapply(df_lclase03$notas, df_lclase03$estudiante, mean)
head(lclase03_mean)
```

```
##      id1      id2      id3      id4      id5      id6
## 3.500000 6.562500 5.687500 5.285714 3.916667 6.000000
```

```
print("Cuasidesviaciones")
```

```
## [1] "Cuasidesviaciones"
```

```
lclase03_sd = tapply(df_lclase03$notas, df_lclase03$estudiante, sd)
head(lclase03_sd)
```

```
##      id1      id2      id3      id4      id5      id6
## 3.500000 2.455860 2.235070 2.721169 3.006936 3.316625
```

Guardamos los objetos

```
save(list = c("lclase03_mean", "lclase03_sd"), file = "calificacionesalumnos.RData")
```

6 Ejercicio 4. Tydiverse, dplyr

Importe el siguiente fichero de datos en R desde la siguiente url: “<https://destio2008.us.es/calvo/descargas/datosVentas.RData>”

```
set.seed(123)
load("datosVentas.RData")
```

El dataset “datosVentas”, tiene las siguientes columnas: “Id, IdCliente, IdVendedor, Dia, Mes, Ano, ValorVendido, IdProducto, IdCategoriaProducto”

Responda a las siguientes cuestiones usando el sistema tidyverse:

6.1 ¿Cuál fue el vendedor que hizo más ventas en total?

```
datosVentas %>%
  group_by(IdVendedor) %>%
  summarise(TotalVendido = sum(ValorVendido, na.rm = TRUE)) %>%
  slice_max(TotalVendido, n = 1)
```

```
## # A tibble: 1 x 2
##   IdVendedor TotalVendido
##       <int>         <dbl>
## 1         37         636528.
```

6.2 ¿En qué 3 categorías de producto se vendió más?

```
datosVentas %>%
  group_by(IdProducto) %>%
  summarise(TotalVendido = sum(ValorVendido, na.rm = TRUE)) %>%
  slice_max(TotalVendido, n = 3)
```

```
## # A tibble: 3 x 2
##   IdProducto TotalVendido
##   <int>      <dbl>
## 1         4      2082808.
## 2        14      2028507.
## 3        17      2022252.
```

6.3 ¿Qué 5 clientes compraron más en el mes de enero de 2020?

```
datosVentas %>%
  filter(Ano == 2020 & Mes == 1) %>%
  group_by(IdCliente) %>%
  summarise(TotalVendido = sum(ValorVendido, na.rm = TRUE)) %>%
  slice_max(TotalVendido, n = 5)
```

```
## # A tibble: 5 x 2
##   IdCliente TotalVendido
##   <int>      <dbl>
## 1        24      27171.
## 2        29      25278.
## 3        27      24558.
## 4        10      24417.
## 5        13      24217.
```

6.4 ¿Qué 10 días del 2020 se vendió más? Mostrarlos de forma descendente, indicando la venta realizada y también el mes al que pertenecían.

```
datosVentas %>%
  filter(Ano == 2020) %>%
  group_by(Dia, Mes) %>%
  summarise(TotalVendido = sum(ValorVendido, na.rm = TRUE)) %>%
  ungroup() %>%
  slice_max(TotalVendido, n = 10) %>%
  arrange(desc(TotalVendido))
```

```
## `summarise()` has grouped output by 'Dia'. You can override using the `.groups`
## argument.
```

```
## # A tibble: 10 x 3
##   Dia   Mes TotalVendido
##   <int> <int>      <dbl>
## 1    17    10      29116.
```

```
## 2      27      7      28689.
## 3       6     10      28571.
## 4      26      5      28427.
## 5       5      5      28235.
## 6       8      9      28190.
## 7       2      9      27951.
## 8      27      9      27907.
## 9      16      8      27862.
## 10     18      1      27760.
```

6.5 Responda a la cuestión “3” sin usar el sistema tidyverse, es decir, usando recursos del sistema base.

```
enero2020 <- datosVentas[datosVentas$Ano == 2020 & datosVentas$Mes == 1,]
totales <- tapply(enero2020$ValorVendido, enero2020$IdCliente, sum, na.rm = TRUE)
ordenados <- sort(totales, decreasing = TRUE)
top5 <- head(ordenados, 5)

top5
```

```
##      24      29      27      10      13
## 27171.36 25278.16 24557.64 24417.01 24217.33
```

Invente 2 cuestiones adicionales sobre este dataset y obtenga la respuesta con el sistema tidyverse.

6.6 ¿Qué 3 categorías de producto son las más vendidas?

```
datosVentas %>%
  group_by(IdCategoriaProducto) %>%
  summarise(TotalVendido = sum(ValorVendido, na.rm = TRUE)) %>%
  slice_max(TotalVendido, n = 3)
```

```
## # A tibble: 3 x 2
##   IdCategoriaProducto TotalVendido
##           <dbl>         <dbl>
## 1                 2      4016467.
## 2                 8      3852395.
## 3                 7      3627416.
```

6.7 ¿Qué producto ha comprado más el cliente que más ha comprado?

```
cliente_top <- datosVentas %>%
  group_by(IdCliente) %>%
  summarise(TotalVendido = sum(ValorVendido, na.rm = TRUE)) %>%
  slice_max(TotalVendido, n = 1) %>%
  pull(IdCliente)
```

```
datosVentas %>%
  filter(IdCliente == cliente_top) %>%
  group_by(IdProducto) %>%
  summarise(NumUnidades = n(), .groups = "drop") %>%
  slice_max(NumUnidades, n = 1)
```

```
## # A tibble: 1 x 2
##   IdProducto NumUnidades
##       <int>         <int>
## 1         17          263
```

7 Ejercicio 5. Análisis y visualización de datos agrupados.

7.1 Instrucciones.

Construya un fichero Rmarkdown (Rmd) comentado, denominado nombredelalumno.Rmd (junto al .html y .pdf, resultantes) en el que se dé respuesta a cada una de las tareas a realizar que se plantean a continuación. Envíe los tres archivos (.Rmd, .html, .pdf) y los ficheros auxiliares generados (se recomienda incluir todos los ficheros en una carpeta con todo el material) en un fichero zip (“nombredelalumno.zip”) mediante “Mensajes de Curso” al profesor Antonio Beato Moreno. Para facilitar la corrección, el fichero ha de tener la misma estructura de secciones y subsecciones que las preguntas de las preguntas que se formulan a continuación.

7.2 Conjuntos de datos.

Conjunto de datos para el estudio: fichero: diabetes_dataset.csv. Columnas:

- Embarazos: Número de veces que la paciente ha estado embarazada.
- Glucosa: Concentración de glucosa plasmática después de una prueba de tolerancia a la glucosa oral de 2 horas.
- Presión arterial: Presión arterial diastólica (mm Hg).
- Espesor de la piel: Espesor del pliegue cutáneo del tríceps (mm).
- Insulina: Insulina sérica a las 2 horas (mu U/ml).
- IMC: Índice de masa corporal (peso en kg/(altura en m2).
- Función DiabetesPedigree: Función que representa el pedigrí de diabetes de la paciente (es decir, probabilidad de diabetes según los antecedentes familiares).
- Edad: edad del paciente (años).
- Resultado: resultado binario (0 o 1) donde 1 indica la presencia de diabetes y 0 indica la ausencia.

7.3 Cargue los datos en R en un dataset. Muestre los 7 primeros casos de los datos. Guarde el dataset en un fichero RData: “Datos_tema2.RData”.

```
data_relacion_2 = read.csv("diabetes_dataset.csv")
head(data_relacion_2,7)
```

```
##   Pregnancies Glucose BloodPressure SkinThickness Insulin  BMI
## 1           6    148           72             35         0 33.6
## 2           1     85           66             29         0 26.6
```



```
## 3      8      183      64      0      0 23.3
## 4      1       89      66     23     94 28.1
## 5      0      137      40     35    168 43.1
## 6      5      116      74      0      0 25.6
## 7      3       78      50     32     88 31.0
## DiabetesPedigreeFunction Age Outcome
## 1      0.627  50      1
## 2      0.351  31      0
## 3      0.672  32      1
## 4      0.167  21      0
## 5      2.288  33      1
## 6      0.201  30      0
## 7      0.248  26      1
```

```
save(data_relacion_2, file = "Datos_tema2.RData")
```

7.4 Análisis exploratorio de los datos

7.4.1 Calcule la tabla de frecuencias (absolutas, relativas y acumuladas) las variables glucosa e IMC, agrupándolas previamente en intervalos. Las tablas deben tener un máximo de 8 intervalos.

```
attach(data_relacion_2)

glucosa_agrupado <- cut(Glucose, breaks = 8)
imc_agrupado <- cut(BMI, breaks = 8)

table_of_frequencies <- function(data_vector){

  tabla <- table(data_vector)

  frecuencias <- data.frame(
    n_i = as.numeric(tabla),
    N_i = cumsum(tabla),
    f_i = as.numeric(round(prop.table(tabla), 3))
  )

  df <- rownames_to_column(frecuencias, var = "Intervalos")

  return(df)}

table_of_frequencies(glucosa_agrupado)

##      Intervalos n_i N_i  f_i
## 1 (-0.199,24.9]   5   5 0.007
## 2 (24.9,49.8]    1   6 0.001
## 3 (49.8,74.6]   22  28 0.029
## 4 (74.6,99.5]  169 197 0.220
## 5 (99.5,124]   260 457 0.339
## 6 (124,149]   168 625 0.219
## 7 (149,174]    86 711 0.112
## 8 (174,199]    57 768 0.074
```

```
table_of_frequencies(imc_agrupado)
```

```
##      Intervalos n_i N_i  f_i
## 1 (-0.0671,8.39]  11  11 0.014
## 2  (8.39,16.8]    0  11 0.000
## 3  (16.8,25.2]   115 126 0.150
## 4  (25.2,33.5]   324 450 0.422
## 5  (33.5,41.9]   243 693 0.316
## 6  (41.9,50.3]    67 760 0.087
## 7  (50.3,58.7]     6 766 0.008
## 8  (58.7,67.2]     2 768 0.003
```

7.4.2 Obtenga una tabla formateada para latex con ayuda del paquete knitr.

```
knitr::kable(
  table_of_frequencies(glucosa_agrupado),
  format = "latex",
  booktabs = TRUE,
  caption = "Table of frequencies",
  label = "tab:freq"
)
```

7.4.3 Obtenga una tabla formateada para html con ayuda del paquete kableExtra.

```
df <- table_of_frequencies(glucosa_agrupado)
df %>%
  kable(format = "html",
        caption = "Table of frequencies") %>%
  kable_styling(
    bootstrap_options = c("striped", "hover", "condensed"),
    full_width = FALSE,
    position = "center"
  ) %>%
  add_header_above(c(" " =1, "Absolute frequencies"=2, "Relative frequencies"=1)) %>%
  column_spec(1, bold = TRUE)
```

7.4.4 Calcule las medidas estadísticas vistas para las variables Presión arterial y Función DiabetesPedigree. Aplique también las funciones de resumen descriptivo, así como el cálculo de las medidas de concentración.

Aplicamos en primer lugar la función summary nativa de R.

```
summary(data_relacion_2[,c("BloodPressure", "DiabetesPedigreeFunction")])
```

```
## BloodPressure    DiabetesPedigreeFunction
## Min.   : 0.00    Min.   :0.0780
## 1st Qu.: 62.00    1st Qu.:0.2437
## Median : 72.00    Median :0.3725
```

```
## Mean : 69.11 Mean :0.4719
## 3rd Qu.: 80.00 3rd Qu.:0.6262
## Max. :122.00 Max. :2.4200
```

Ahora definimos una función propia para ampliar estas estadísticas sobre los datos.

```
statistical_summary<- function(data){
  n = length(data)
  range = max(data)-min(data)

  cuasivar = var(data)
  varianza = cuasivar*((n-1)/n)
  standar_deviation = sd(data)

  tabla <- table(data)
  ux <- unique(data)
  ni <- tabulate(match(data, ux))
  mode <- ux[which.max(ni)]

  gini <- gini(ux,ni)

  percentiles <- quantile(data, probs = seq(0, 1, 0.01),names = TRUE, type = 2)

  skewness <- skewness(data)
  kurtosis <- kurtosis(data)
  results <- data.frame(
    n = n,
    rango = range,
    cuasivar = cuasivar,
    varianza = varianza,
    desviacion_tipica = standar_deviation,
    moda = mode,
    asimetria = skewness,
    kurtosis = kurtosis,
    gini = gini
  )
  print(results)
  print(percentiles)
  return(results)
}
```

Aplicamos la función.

```
statistical_summary(BloodPressure)
```

```
##      n rango cuasivar varianza desviacion_tipica moda asimetria kurtosis
## 1 768 122 374.6473 374.1594 19.35581 70 -1.836413 5.11751
##      gini
## 1 0.1359317
##  0%   1%   2%   3%   4%   5%   6%   7%   8%   9%  10%  11%  12%  13%  14%  15%
##   0    0    0    0    0  38  48  50  50  52  54  54  56  56  58  58
## 16% 17% 18% 19% 20% 21% 22% 23% 24% 25% 26% 27% 28% 29% 30% 31%
##  60  60  60  60  60  62  62  62  62  62  64  64  64  64  64  65
```

```
## 32% 33% 34% 35% 36% 37% 38% 39% 40% 41% 42% 43% 44% 45% 46% 47%
## 66 66 66 66 68 68 68 68 68 68 70 70 70 70 70 70
## 48% 49% 50% 51% 52% 53% 54% 55% 56% 57% 58% 59% 60% 61% 62% 63%
## 70 72 72 72 72 72 72 74 74 74 74 74 74 74 75 76
## 64% 65% 66% 67% 68% 69% 70% 71% 72% 73% 74% 75% 76% 77% 78% 79%
## 76 76 76 76 78 78 78 78 78 78 80 80 80 80 80 82
## 80% 81% 82% 83% 84% 85% 86% 87% 88% 89% 90% 91% 92% 93% 94% 95%
## 82 82 82 84 84 84 85 86 86 88 88 88 88 90 90 90
## 96% 97% 98% 99% 100%
## 92 95 100 106 122
```

```
##      n rango cuasivar varianza desviacion_tipica moda asimetria kurtosis
## 1 768 122 374.6473 374.1594 19.35581 70 -1.836413 5.11751
##      gini
## 1 0.1359317
```

```
statistical_summary(DiabetesPedigreeFunction)
```

```
##      n rango cuasivar varianza desviacion_tipica moda asimetria kurtosis
## 1 768 2.342 0.1097786 0.1096357 0.3313286 0.254 1.912418 5.528539
##      gini
## 1 0.355393
## 0% 1% 2% 3% 4% 5% 6% 7% 8% 9% 10%
## 0.0780 0.0920 0.1180 0.1270 0.1340 0.1400 0.1430 0.1490 0.1540 0.1600 0.1650
## 11% 12% 13% 14% 15% 16% 17% 18% 19% 20% 21%
## 0.1700 0.1780 0.1830 0.1890 0.1920 0.1980 0.2030 0.2070 0.2100 0.2190 0.2270
## 22% 23% 24% 25% 26% 27% 28% 29% 30% 31% 32%
## 0.2330 0.2360 0.2380 0.2435 0.2470 0.2520 0.2540 0.2570 0.2590 0.2610 0.2630
## 33% 34% 35% 36% 37% 38% 39% 40% 41% 42% 43%
## 0.2680 0.2700 0.2780 0.2840 0.2870 0.2920 0.2970 0.3030 0.3050 0.3150 0.3240
## 44% 45% 46% 47% 48% 49% 50% 51% 52% 53% 54%
## 0.3300 0.3370 0.3420 0.3460 0.3560 0.3650 0.3725 0.3810 0.3910 0.3990 0.4030
## 55% 56% 57% 58% 59% 60% 61% 62% 63% 64% 65%
## 0.4120 0.4220 0.4300 0.4390 0.4460 0.4540 0.4660 0.4830 0.4950 0.5010 0.5140
## 66% 67% 68% 69% 70% 71% 72% 73% 74% 75% 76%
## 0.5260 0.5360 0.5450 0.5510 0.5640 0.5820 0.5870 0.5970 0.6100 0.6265 0.6400
## 77% 78% 79% 80% 81% 82% 83% 84% 85% 86% 87%
## 0.6540 0.6660 0.6780 0.6870 0.6980 0.7110 0.7270 0.7380 0.7570 0.7850 0.8130
## 88% 89% 90% 91% 92% 93% 94% 95% 96% 97% 98%
## 0.8310 0.8550 0.8800 0.9050 0.9470 0.9680 1.0570 1.1360 1.1890 1.2580 1.3910
## 99% 100%
## 1.6990 2.4200
```

```
##      n rango cuasivar varianza desviacion_tipica moda asimetria kurtosis
## 1 768 2.342 0.1097786 0.1096357 0.3313286 0.254 1.912418 5.528539
##      gini
## 1 0.355393
```

7.4.5 Estudie e identifique los posibles outliers de la variable Presión arterial. Acompañe con las gráficas adecuadas.

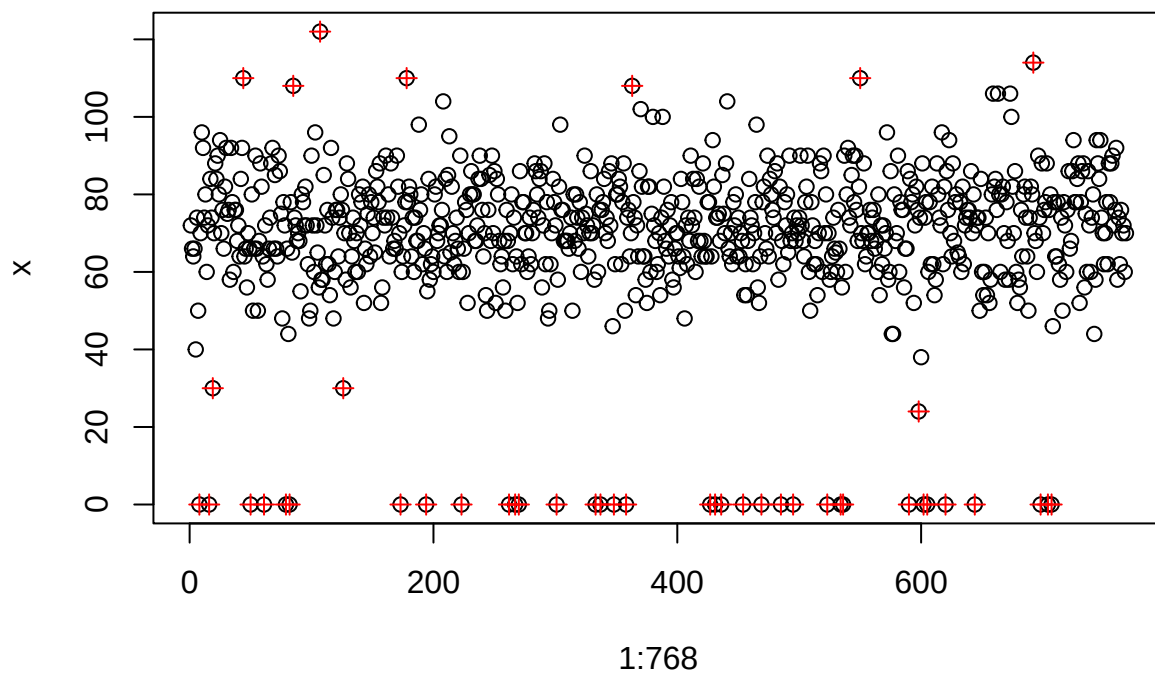
Como convenio tomaremos dos desviaciones típicas.

```
x <- BloodPressure
int <- c(mean(x)-2*sd(x),mean(x)+2*sd(x))
a <- which((x < int[1]) | (x > int[2]))
#outliers:
x[a]
```

```
## [1] 0 0 30 110 0 0 0 0 108 122 30 0 110 0 0 0 0 0 0
## [20] 0 0 0 0 108 0 0 0 0 0 0 0 0 0 0 110 0 24 0
## [39] 0 0 0 114 0 0 0
```

En la salida anterior se muestran los outliers

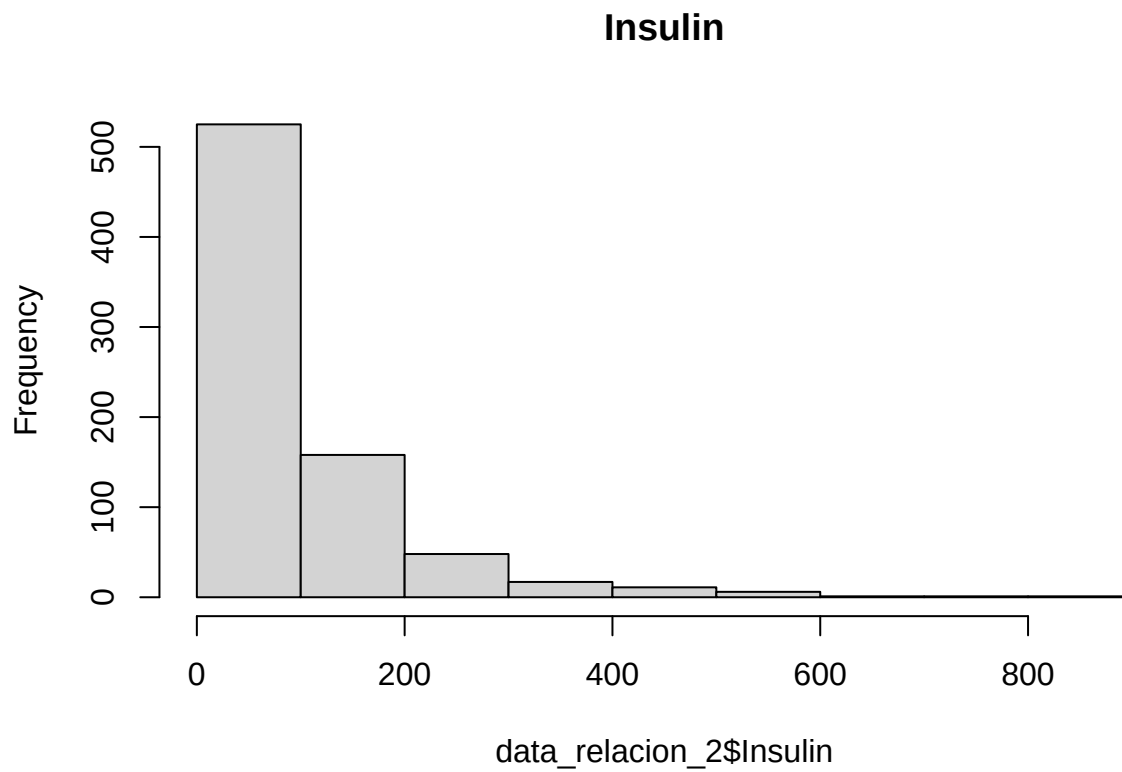
```
plot(1:768,x)
points(a,x[a],col="red",pch=3)
```



En esta gráfica se muestran los valores atípicos resaltados en rojos.

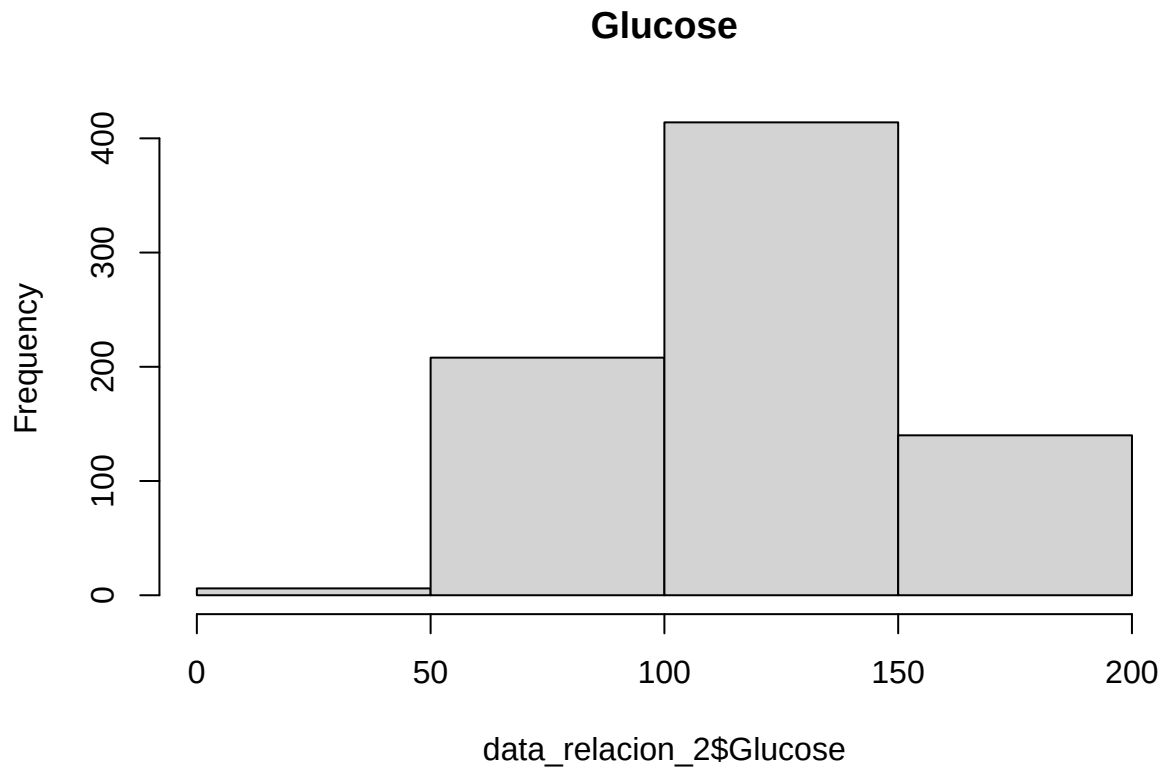
7.4.6 Obtenga el histograma de al menos la variable Insulina.

```
hist(data_relacion_2$Insulin,main = "Insulin")
```



7.4.7 Seleccione la variable Glucosa y agrupe sus valores en cuatro intervalos.

```
hist(data_relacion_2$Glucose, main = "Glucose", breaks = 4)
```



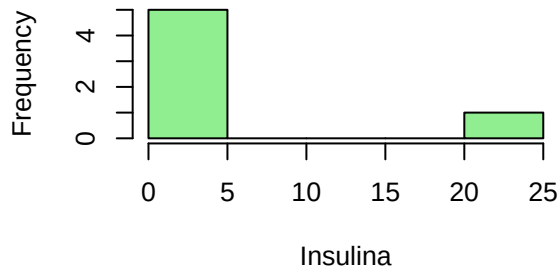
7.4.8 En cada uno de los grupos definidos por los grupos de esa segunda variable, obtenga el histograma de la primera variable y represente los histogramas conjuntamente.

```
#First we add a column to split the dataframe into groups
data_relacion_2$glucosa_grupo <- cut(data_relacion_2$Glucose, breaks = 4)

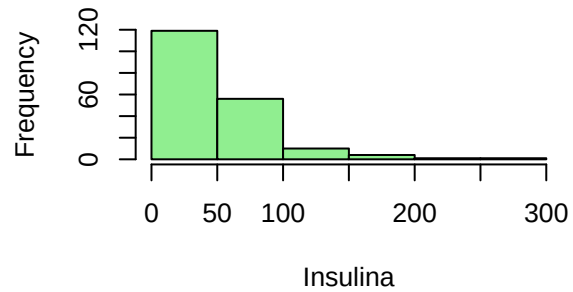
#Secondly we define the 2x2 graphic
par(mfrow = c(2, 2))

#lastly we iterate for creating the subplots
for (g in levels(data_relacion_2$glucosa_grupo)) {
  hist(data_relacion_2$Insulin[data_relacion_2$glucosa_grupo == g],
       main = paste("Insulina para Glucosa en", g),
       xlab = "Insulina",
       col = "lightgreen",
       border = "black")
}
```

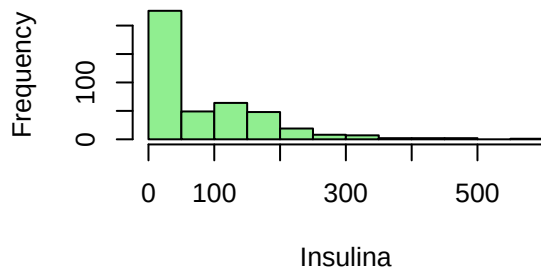
Insulina para Glucosa en $(-0.199,49.8]$



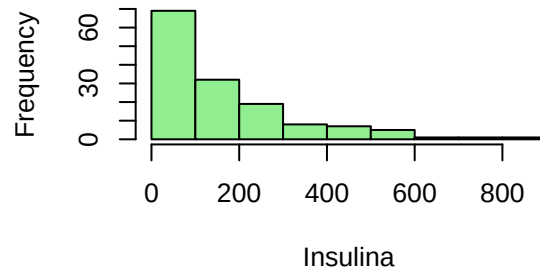
Insulina para Glucosa en $(49.8,99.5]$



Insulina para Glucosa en $(99.5,149]$



Insulina para Glucosa en $(149,199]$



7.4.9 Obtenga la tabla de frecuencias conjuntas de las variables usadas en el apartado anterior (ya agrupadas en intervalos)

```
data_relacion_2$insulina_grupo <- cut(data_relacion_2$Insulin, breaks = 10)
tabla_conjunta<-table(data_relacion_2$insulina_grupo, data_relacion_2$glucosa_grupo)
tabla_conjunta
```

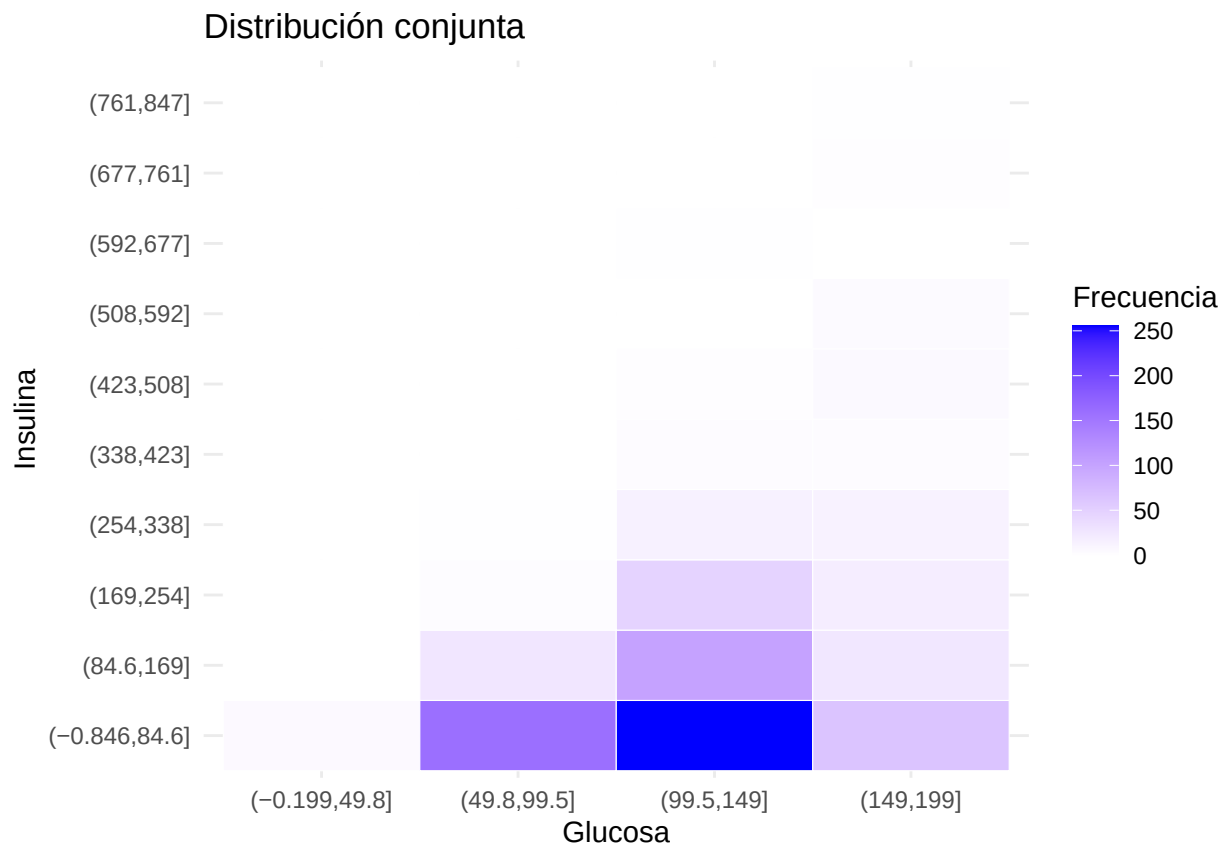
```
##
##          (-0.199,49.8] (49.8,99.5] (99.5,149] (149,199]
## (-0.846,84.6]          6          160          256          65
## (84.6,169]              0           27          102          26
## (169,254]              0            3           48          19
## (254,338]              0            1           15          14
## (338,423]              0            0            4           4
## (423,508]              0            0            2           7
## (508,592]              0            0            0           5
## (592,677]              0            0            1           0
## (677,761]              0            0            0           2
## (761,847]              0            0            0           1
```

7.4.10 Represente las frecuencias conjuntas mediante una representación gráfica adecuada.

Se procede a plantear un tile plot para mostrar la densidad de la distribución conjunta


```
df_tab <- as.data.frame(tabla_conjunta)
colnames(df_tab) <- c("Insulina", "Glucosa", "Freq")

ggplot(df_tab, aes(x = Glucosa, y = Insulina, fill = Freq)) +
  geom_tile(color = "white") +
  scale_fill_gradient(low = "white", high = "blue") +
  labs(title = "Distribución conjunta",
       x = "Glucosa",
       y = "Insulina",
       fill = "Frecuencia") +
  theme_minimal()
```



7.4.11 Seleccione las variables Edad y Espesor de la piel. Agrupar la primera de ellas en 5 intervalos e identifique los posibles outliers de la segunda variable en cada grupo definido por los intervalos de la primera variable. Usando la librería ggplot2, acompañe con las gráficas adecuadas.

```
data_relacion_2$Age_group <- cut(data_relacion_2$Age, breaks = 5)

plots <- list()

outliers <- function(x){
  int <- c(mean(x) - 2*sd(x), mean(x) + 2*sd(x))
```

```

  which(x < int[1] | x > int[2])
}

for (y in levels(data_relacion_2$Age_group)) {

  x <- data_relacion_2$SkinThickness[data_relacion_2$Age_group == y]

  df <- data.frame(
    valor = x,
    is_outlier = rep(NA, length(x))
  )

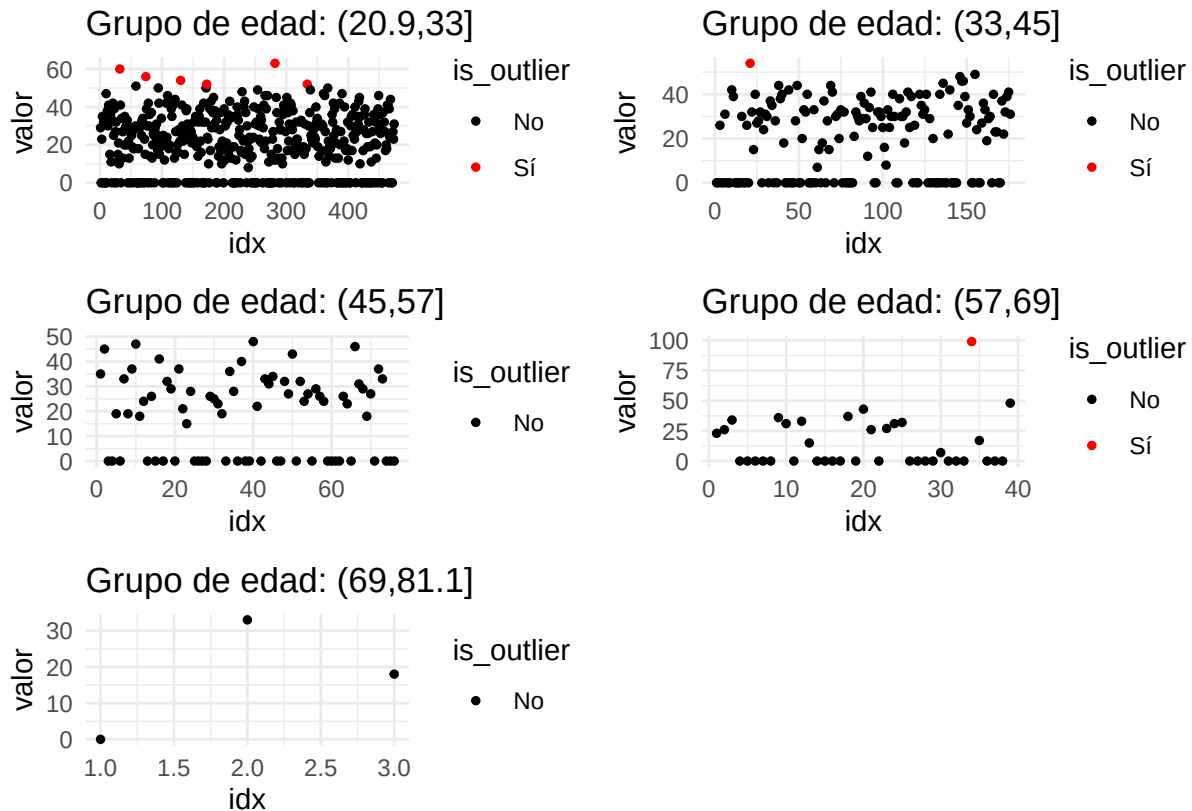
  a <- outliers(x)

  df$is_outlier[a] = "Sí"
  df$is_outlier[is.na(df$is_outlier)] = "No"
  df$idx <- seq_len(nrow(df))

  plots[[y]] <- ggplot(df, aes(x = idx, y = valor, color = is_outlier)) +
    geom_point(size = 1) +
    scale_color_manual(values = c("No" = "black", "Sí" = "red")) +
    labs(title = paste("Grupo de edad:", y)) +
    theme_minimal()
}

# Afficher les 4 graphiques ensemble
wrap_plots(plots, ncol = 2)

```



8 Ejercicio 6. Ajuste lineal

8.1 Genere 200 valores de una distribución $N(0,1)$ tomando como semilla 1125. Guardelos resultados en un vector X.

```
set.seed(1125)
```

```
X = rnorm(n = 200, mean = 0, sd = 1)
X
```

```
## [1] -0.086764479  0.629840317 -1.369137799  1.065127903 -0.264724900
## [6]  0.512874452  0.434830847  0.883365131 -1.772419973  0.366597635
## [11] -1.876896720 -0.033481876  1.712831358  0.642344036 -0.280173269
## [16]  1.609997727  0.763391902  1.612552878 -0.457246923 -0.936258537
## [21]  1.805748328  0.464722827  1.033247327 -2.060093231 -0.948938606
## [26]  1.308190397 -0.168748955  1.790301942  0.996651533  1.668158748
## [31] -0.259852102  1.568320759 -0.217878238  0.055107898  2.381588026
## [36] -1.585079895  0.493532831  0.395641917  0.666759353  0.902657276
## [41] -2.656213144  1.474166656 -0.066802562  1.282320866 -0.743808028
## [46] -0.721774445  0.594550886  0.650352655  0.993723373  1.021569491
## [51]  0.031444495  1.144040633  0.229471447  1.074340884 -0.082949397
## [56]  0.926573673 -1.930102330 -0.775308988 -0.311728023  3.069914647
## [61] -0.094252176 -0.035497772  0.278288864 -3.630353742  0.604511562
```

```
## [66] 0.371701491 0.152322478 1.090419823 0.858258800 -0.691007001
## [71] 0.812079196 -1.360193616 -0.534461362 0.529403816 -0.126240458
## [76] 0.244666725 -1.506646018 -0.847483290 -1.017092287 -0.365970954
## [81] 1.132953281 1.940129908 1.339656297 -0.435853886 0.876266366
## [86] 1.420917843 -0.960745121 -2.312462429 0.497533583 -0.467749692
## [91] -1.005663321 0.075772083 -0.179899246 -0.353119291 -0.007461425
## [96] 1.678189423 0.199330179 -0.356477154 -1.131130158 0.102000893
## [101] -1.269687022 0.063814456 0.230829880 0.224313892 -0.739659633
## [106] 0.489980237 1.591589133 -0.430765954 0.976775194 -1.084522521
## [111] 2.233823852 -0.045622268 -0.680267034 1.190308063 0.730939691
## [116] 0.012492030 0.360453770 -0.086257805 2.355706444 -0.219765733
## [121] 1.688080365 2.084978002 0.239318551 1.119500885 0.918076202
## [126] -1.500457104 2.296874928 -0.221973224 1.201066450 1.069788431
## [131] 0.401594901 0.267878937 -0.574698232 -0.904132848 0.210723753
## [136] 0.404538203 1.295096035 1.197406070 -0.984889421 0.398804469
## [141] 0.357619459 0.697620768 -0.166072865 1.675906254 -0.100562231
## [146] 0.744438386 -0.626539336 -0.724918983 -0.850825968 -0.056461444
## [151] 1.078587337 1.372951513 -0.320232564 -2.154058078 -2.688728718
## [156] -1.091402369 1.015788142 1.216050176 1.138071319 0.763769104
## [161] -0.705011250 1.465501122 -1.083510420 0.821318377 -0.279926283
## [166] 1.161500860 0.410907791 0.638903053 -1.031345571 -1.404188154
## [171] -1.003886541 -0.520366061 1.308849040 0.162498063 0.789537729
## [176] 0.328910947 -0.073690468 -1.121182822 -0.114206712 -1.759509945
## [181] 0.208468560 0.261814319 1.200567915 0.349242509 -2.056882686
## [186] 0.594594093 -0.960883810 1.150529093 0.855426635 -0.074480115
## [191] -0.751894313 -0.110253180 0.999192126 -1.233133701 0.147439356
## [196] -1.000260832 0.760045878 -0.160787414 0.715183189 -0.903238266
```

8.2 Genere 200 valores de una distribución normal de media cero y varianza 0.36 . Guarde los resultados en un vector ϵ .

```
epsilon = rnorm(n = 200, mean = 0, sd = 0.6)
epsilon
```

```
## [1] 0.472947956 0.138781111 -0.247664098 0.869387825 0.387690862
## [6] -0.274091675 0.081709110 -0.556697941 0.112377049 -0.153912365
## [11] 0.319026026 0.518203395 0.518759718 -0.504475955 -0.761181411
## [16] 0.490388931 0.730875177 -0.148383138 0.961815272 -0.083204589
## [21] 0.670322751 0.156095059 0.607327037 -0.468140228 -1.157197744
## [26] -0.392734334 -0.524813783 -0.891058019 -0.429358281 -1.051142284
## [31] -1.114111440 0.200139029 0.206778993 -0.103897947 -0.781343157
## [36] -0.489142820 -0.274913679 0.212177775 -0.228445900 -0.545559627
## [41] -0.242452887 -0.746896967 0.039129249 -0.303634235 -0.814166490
## [46] -0.567053892 -0.034883043 -1.094315444 0.707999860 -0.195374937
## [51] 0.322179856 -1.142399228 -0.479014111 0.699233319 0.028935843
## [56] -0.022808843 -0.777822435 -0.182660107 0.540018720 1.098173432
## [61] 1.205172788 0.818667934 -0.014217999 -0.315227393 0.120728634
## [66] -0.450482758 -0.766623631 -0.216050549 -0.059861452 -1.105744863
## [71] 0.577385052 -0.377961648 0.063618474 1.022840630 -0.050479870
## [76] -0.582131906 -0.801298025 1.038500318 -0.545752761 -0.472350716
## [81] 1.955079628 -0.826448778 0.765718577 -0.259450528 0.325094319
## [86] -0.180706869 -1.294941272 -0.485337934 0.756316396 0.068683206
```

```
## [91] 1.456871275 -0.366246813 0.902454430 0.371292560 0.160684948
## [96] 0.745399901 -0.483429882 0.653936483 -0.716031831 -0.337383230
## [101] -0.618938391 0.499777872 0.123986305 -0.343712703 -0.231211452
## [106] -0.356864064 -0.473621092 -0.466710448 -0.027217993 0.070022408
## [111] -0.383792994 0.166451677 -1.076000845 1.089797230 -0.759969152
## [116] -0.828228899 0.060364027 -0.037824955 0.429205793 0.293146579
## [121] -0.081868643 -0.150085393 0.461435443 -0.375746579 0.070964327
## [126] 0.200265905 0.346943712 0.276553099 -0.999043247 -1.427024587
## [131] 0.570445211 0.753186752 0.758034108 -0.442878053 -0.066730495
## [136] -0.563944897 -0.058763600 -1.081902014 0.930032277 -0.011702066
## [141] 0.141217948 -0.260163480 0.571326896 -0.138376873 0.030789056
## [146] -0.313523052 -1.539260619 -0.474222424 0.238185615 0.648359847
## [151] -0.177733286 -0.194917633 -1.098909589 0.278840541 -0.933465507
## [156] -0.798597985 0.006368916 0.164139960 0.157281426 0.499865818
## [161] 0.486481584 -0.580771602 -1.096679871 0.051636195 -0.212046189
## [166] 0.032330799 -0.762538135 0.824281801 -0.435030357 -1.522703016
## [171] -0.196053693 -1.029384419 -0.042657565 -1.425254472 0.259014272
## [176] 1.290317904 0.665601463 0.674031845 0.704287990 -0.487293217
## [181] 1.046269113 -0.124472578 -0.804353618 -0.554126796 0.583748143
## [186] 0.985514067 -0.050679913 0.059436854 0.480929050 -0.379152635
## [191] -0.552327177 0.561276934 -0.223435242 0.883031672 -0.508013180
## [196] 0.004782473 0.797785565 -0.603384666 -0.174606777 0.922315036
```

Usamos 0.6 en el parámetro sd porque es la desviación estándar o típica. Por ello, como $0.6 = \sqrt{0.36} = \sqrt{\sigma^2}$ ponemos 0.6 en el parámetro.

8.3 Utilizando los vectores X y epsilon, obtenga un vector Y descrito por el modelo $Y = -1 + 0.5X + \epsilon$ ¿Qué longitud tiene el vector Y? ¿Cuáles son los valores de β_0 y β_1 en este modelo?

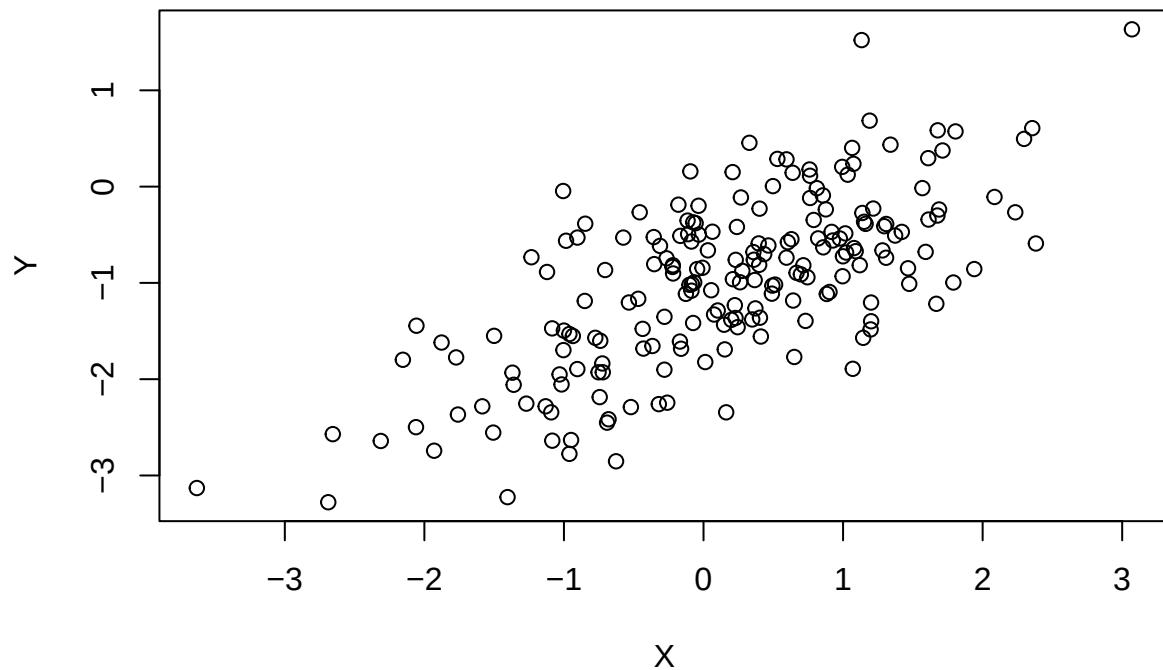
```
Y = -1 + 0.5*X + epsilon
length(Y)
```

```
## [1] 200
```

Tiene longitud 200. Los valores β_0 y β_1 son -1 y 0.5 respectivamente, dado el diseño de la variable Y.

8.4 Considere los datos, $\{(x_i, y_i)\}$, realice un gráfico para mostrar la relación entre estos valores.

```
plot(X,Y)
```



Es clara la relación lineal directa que se estableció en la variable Y con X

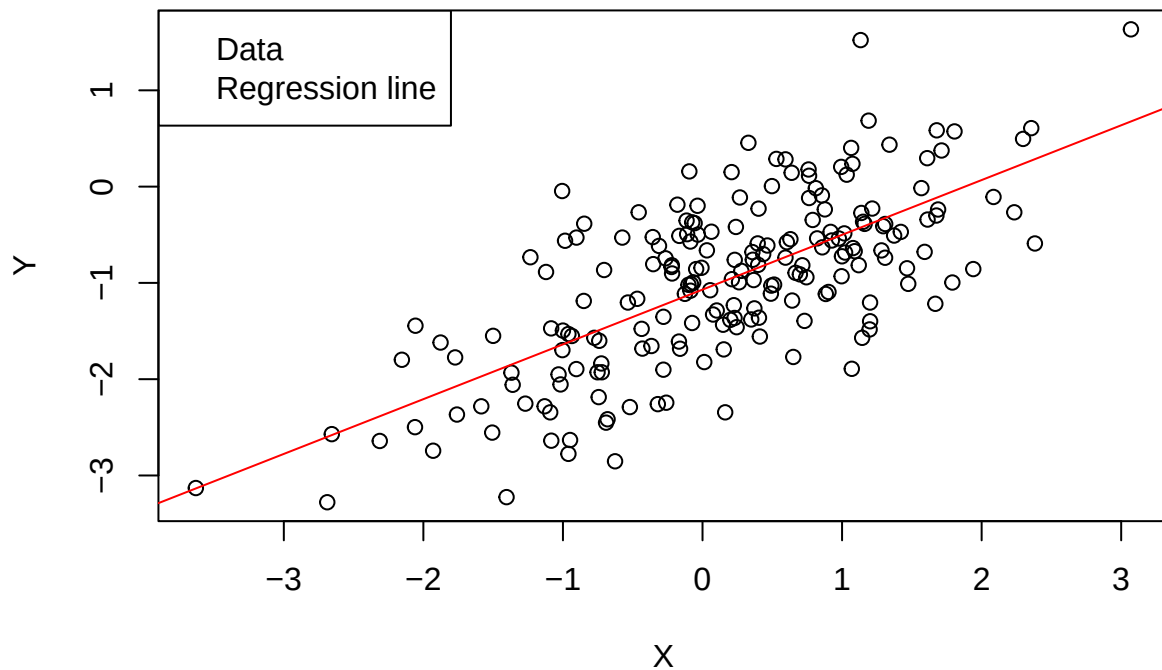
8.5 Ajuste un modelo de regresión lineal

```
model = lm(Y~X)
model
```

```
##
## Call:
## lm(formula = Y ~ X)
##
## Coefficients:
## (Intercept)          X
##      -1.0697       0.5688
```

8.6 Represente el modelo

```
plot(X,Y)
abline(a=coef(model)["(Intercept)"],b=coef(model)["X"], col = "red")
legend("topleft",legend = c("Data", "Regression line"))
```



8.7 Comente los resultados obtenidos (compare $\hat{\beta}_0$, $\hat{\beta}_1$ y $\hat{\sigma}^2$ con β_0 , β_1 y σ^2).

```
model$coefficients
```

```
## (Intercept)      X
##  -1.069748    0.568810
```

Estos son los valores estimados de β_0 y β_1 los cuales se asemejan muchísimo a los valores reales (-1 y 0.5).

```
summary(model)$sigma^2
```

```
## [1] 0.3986758
```

Esta es la estimación de σ que también se asemeja mucho a la real de 0.36.

9 Ejercicio 7. Ajuste lineal por subconjuntos. Selección de modelo. Evaluación de modelos.

Considere el conjunto de datos Boston disponible en el paquete de R, ISLR2. Se desea discutir diferentes modelos de regresión lineal para explicar la variable `nox` a partir de las restantes variables disponibles en el conjunto de datos, para ello responda a los siguientes apartados.

```
data(Boston)
```

9.1 Obtenga el mejor modelo con 3 variables predictoras utilizando regsubsets. Lo denotaremos por M3. Incluya y comente alguno de los gráficos que pueden obtenerse con regsubsets.

```
subconjunto=leaps::regsubsets(nox~., data=Boston, nvmax=3, nbest=1)
subconjunto
```

```
## Subset selection object
## Call: regsubsets.formula(nox ~ ., data = Boston, nvmax = 3, nbest = 1)
## 13 Variables (and intercept)
##           Forced in Forced out
## crim          FALSE      FALSE
## zn             FALSE      FALSE
## indus          FALSE      FALSE
## chas           FALSE      FALSE
## rm             FALSE      FALSE
## age            FALSE      FALSE
## dis            FALSE      FALSE
## rad            FALSE      FALSE
## tax            FALSE      FALSE
## ptratio        FALSE      FALSE
## black          FALSE      FALSE
## lstat          FALSE      FALSE
## medv           FALSE      FALSE
## 1 subsets of each size up to 3
## Selection Algorithm: exhaustive
```

```
summary(subconjunto)
```

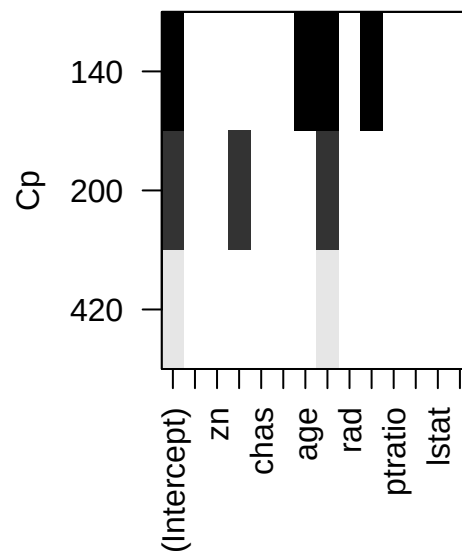
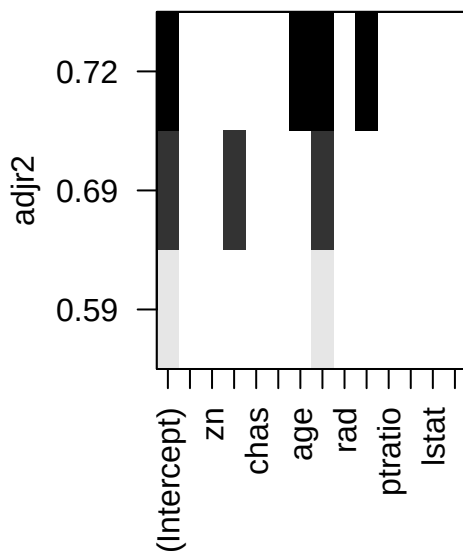
```
## Subset selection object
## Call: regsubsets.formula(nox ~ ., data = Boston, nvmax = 3, nbest = 1)
## 13 Variables (and intercept)
##           Forced in Forced out
## crim          FALSE      FALSE
## zn             FALSE      FALSE
## indus          FALSE      FALSE
## chas           FALSE      FALSE
## rm             FALSE      FALSE
## age            FALSE      FALSE
## dis            FALSE      FALSE
## rad            FALSE      FALSE
## tax            FALSE      FALSE
## ptratio        FALSE      FALSE
## black          FALSE      FALSE
## lstat          FALSE      FALSE
## medv           FALSE      FALSE
## 1 subsets of each size up to 3
## Selection Algorithm: exhaustive
```



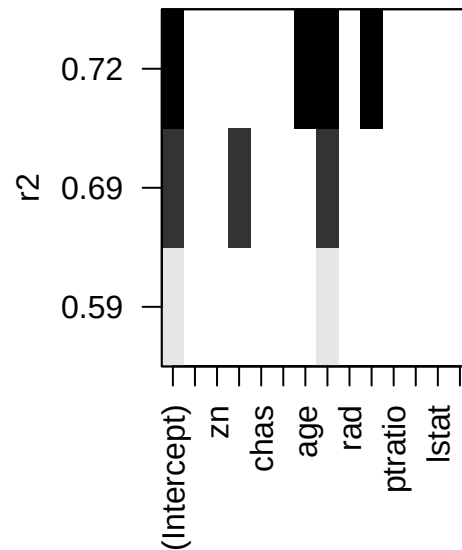
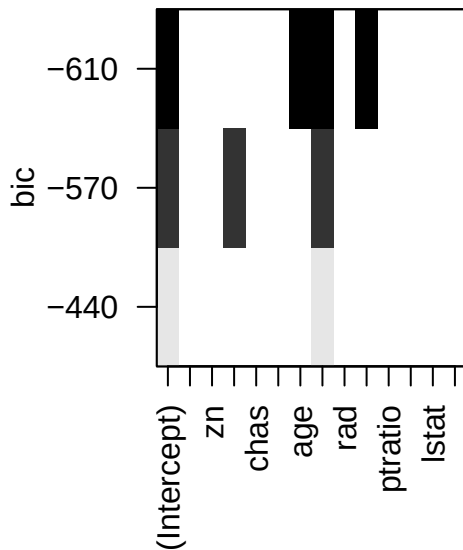
```
##          crim zn   indus chas rm  age dis rad tax ptratio black lstat medv
## 1  ( 1 ) " "   " "   " "   " "   " "   "*" " "   " "   " "   " "   " "
## 2  ( 1 ) " "   " "   "*"   " "   " "   " "   "*" " "   " "   " "   " "
## 3  ( 1 ) " "   " "   " "   " "   " "   "*"   "*" " "   "*" " "   " "   " "
```

```
par(mfrow = c(1, 2))

plot(subconjunto,scale="adjr2")
plot(subconjunto,scale="Cp")
```



```
par(mfrow = c(1, 2))
plot(subconjunto,scale="bic")
plot(subconjunto,scale="r2")
```



El mejor modelo con tres variables explicativas es el formado por las variables age, dis, tax. Fijándonos en el primer gráfico por ejemplo, vemos el R^2 ajustado. Se observa que la parte explicada por el modelo pues aumenta con la introducción de estas tres variables, es decir, el modelo de 3 variables es mejor que el de dos y el de uno. Ciertamente, que la ganancia entre el modelo de 3 variables y dos es de 0.03, podría ser interesante tomar el modelo de dos variables por ser más simple. El coeficiente marca 0.72 que no es un ajuste muy bueno.

```
# store the model as m3
M3=lm(nox~age+dis+tax, data=Boston)
```

9.2 Ecuación del modelo M3, interpretación de los coeficientes. Tests y medidas de bondad de ajuste de este modelo.

```
summary(M3)
```

```
##
## Call:
## lm(formula = nox ~ age + dis + tax, data = Boston)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.119345 -0.034016 -0.006181  0.024003  0.257252
##
## Coefficients:
```

```
##           Estimate Std. Error t value Pr(>|t|)
## (Intercept)  4.689e-01  1.858e-02  25.232 < 2e-16 ***
## age         1.142e-03  1.500e-04   7.609 1.38e-13 ***
## dis        -2.156e-02  2.046e-03 -10.537 < 2e-16 ***
## tax         2.188e-04  1.968e-05  11.116 < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.06185 on 502 degrees of freedom
## Multiple R-squared:  0.7168, Adjusted R-squared:  0.7151
## F-statistic: 423.5 on 3 and 502 DF,  p-value: < 2.2e-16
```

Los estimates nos proporcionan los coeficientes para el modelo, al ser un modelo lineal múltiple la ecuación sería la siguiente: $y = 4.689e-01 + 1.142e-03 \cdot age - 2.156e-02 \cdot dis + 2.188e-04 \cdot tax$, donde y es la variable respuesta, y en este caso sería nox. Interpretaremos, a modo de ejemplo, el coeficiente de age. Por cada unidad que aumente edad, se predice que el impacto que va tener en la variable respuesta es de 1.142e-03. En cuanto a los contrastes fundamentales vemos que todas las variables aparecen con los tres asteriscos que indican un p-valor asociado al contraste menor a 0.001, luego todos son significativamente distintos de cero, lo que quiere decir que todas las variables consideradas en el modelo deben mantenerse porque aportan explicabilidad. Vemos que el R^2 múltiple es de 0.72, como ya calculaba regsubsets, prácticamente idéntico es el valor del R^2 ajustado. Es un modelo medianamente bueno. También vemos una desviación típica de 0.06 en los residuos, lo que quiere decir que en promedio las estimaciones se desvían esa cantidad del valor real. Por último, como el p-valor del modelo es $< 2.2e-16$ entonces el modelo es significativo.

9.3 En M3, estimación del valor medio de nox.

```
mean(predict(M3))
```

```
## [1] 0.5546951
```

9.4 Halle el intervalo de confianza y la predicción para este valor para los valores (70, 4, 310). Muestre los resultados en una tabla.

```
new_obs <- data.frame(
  age = 70,
  dis = 4,
  tax = 310
)
results_table <- predict(M3, newdata = new_obs, interval = "confidence", level = 0.95)
results_table <- data.frame(
  results_table,
  mean_nox_value = mean(predict(M3))
)
results_table
```

```
##           fit          lwr          upr mean_nox_value
## 1 0.5304112 0.5237909 0.5370314          0.5546951
```

El primer valor es la predicción para la muestra dada, los siguientes dos los extremos del intervalo de confianza para un nivel de significación del 5% y por último la estimación del valor medio de nox.

9.5 Justifica la validez de la predicción.

```
shapiro.test(M3$residuals)
```

```
##  
## Shapiro-Wilk normality test  
##  
## data:  M3$residuals  
## W = 0.86808, p-value < 2.2e-16
```

Rechazamos la hipótesis de normalidad sobre la distribución de los residuos por lo que el modelo lineal no sería adecuado ya que las hipótesis del modelo no se cumplen. Particularmente, al no cumplirse la hipótesis de normalidad esto afecta a la validez de los intervalos de confianza, pues la distribución del estadístico usa principalmente la hipótesis sobre la normalidad de los residuos.

9.6 Métodos de selección de variables paso a paso. Aplica el método de selección de variables "both". Comenta los resultados obtenidos, y escribe el modelo final, Mf, que propondrías.

```
#modelo constante  
null=lm(nox~1, data=Boston)  
  
# Modelo saturado  
full=lm(nox~., data=Boston)  
  
MF=stepAIC(null, scope=list(upper=full), direction="both", criterion=AIC, trace=1)
```

```
## Start:  AIC=-2180.08  
## nox ~ 1  
##  
##           Df Sum of Sq  RSS    AIC  
## + dis      1    4.0124 2.7686 -2631.3  
## + indus    1    3.9544 2.8265 -2620.9  
## + age      1    3.6281 3.1528 -2565.6  
## + tax      1    3.0260 3.7549 -2477.2  
## + rad      1    2.5351 4.2458 -2415.0  
## + lstat    1    2.3675 4.4135 -2395.4  
## + zn       1    1.8097 4.9713 -2335.2  
## + medv     1    1.2382 5.5427 -2280.1  
## + crim     1    1.2017 5.5793 -2276.8  
## + black    1    0.9794 5.8015 -2257.0  
## + rm       1    0.6192 6.1617 -2226.5  
## + ptratio  1    0.2420 6.5389 -2196.5  
## + chas     1    0.0564 6.7246 -2182.3  
## <none>             6.7810 -2180.1  
##  
## Step:  AIC=-2631.35  
## nox ~ dis  
##
```

```

##          Df Sum of Sq    RSS    AIC
## + indus   1    0.6522  2.1163 -2765.3
## + tax     1    0.6266  2.1420 -2759.2
## + rad     1    0.4790  2.2896 -2725.5
## + medv    1    0.3997  2.3689 -2708.2
## + lstat   1    0.3918  2.3768 -2706.6
## + age     1    0.3753  2.3932 -2703.1
## + black   1    0.1799  2.5887 -2663.3
## + rm      1    0.1474  2.6211 -2657.0
## + crim    1    0.1317  2.6369 -2654.0
## <none>                2.7686 -2631.3
## + chas    1    0.0015  2.7670 -2629.6
## + ptratio 1    0.0007  2.7678 -2629.5
## + zn      1    0.0004  2.7682 -2629.4
## - dis     1    4.0124  6.7810 -2180.1
##
## Step:  AIC=-2765.29
## nox ~ dis + indus
##
##          Df Sum of Sq    RSS    AIC
## + age     1    0.18249  1.9338 -2808.9
## + rad     1    0.16263  1.9537 -2803.8
## + tax     1    0.16064  1.9557 -2803.2
## + lstat   1    0.10395  2.0124 -2788.8
## + medv    1    0.09118  2.0252 -2785.6
## + black   1    0.06272  2.0536 -2778.5
## + ptratio 1    0.05888  2.0574 -2777.6
## + crim    1    0.03881  2.0775 -2772.7
## + rm      1    0.01058  2.1058 -2765.8
## <none>                2.1163 -2765.3
## + zn      1    0.00614  2.1102 -2764.8
## + chas    1    0.00225  2.1141 -2763.8
## - indus   1    0.65224  2.7686 -2631.3
## - dis     1    0.71022  2.8266 -2620.9
##
## Step:  AIC=-2808.92
## nox ~ dis + indus + age
##
##          Df Sum of Sq    RSS    AIC
## + rad     1    0.14388  1.7900 -2846.0
## + tax     1    0.14064  1.7932 -2845.1
## + ptratio 1    0.07285  1.8610 -2826.3
## + black   1    0.05561  1.8782 -2821.7
## + medv    1    0.04646  1.8874 -2819.2
## + lstat   1    0.03567  1.8982 -2816.3
## + crim    1    0.02898  1.9049 -2814.6
## + zn      1    0.01722  1.9166 -2811.4
## <none>                1.9338 -2808.9
## + rm      1    0.00661  1.9272 -2808.7
## + chas    1    0.00145  1.9324 -2807.3
## - age     1    0.18249  2.1163 -2765.3
## - dis     1    0.22930  2.1631 -2754.2
## - indus   1    0.45940  2.3932 -2703.1
##

```

```

## Step: AIC=-2846.04
## nox ~ dis + indus + age + rad
##
##      Df Sum of Sq  RSS    AIC
## + ptratio  1  0.173983 1.6160 -2895.8
## + medv     1  0.026098 1.7639 -2851.5
## + black    1  0.016076 1.7739 -2848.6
## + lstat    1  0.014991 1.7750 -2848.3
## + zn       1  0.009600 1.7804 -2846.8
## + tax      1  0.008710 1.7812 -2846.5
## + rm       1  0.007927 1.7820 -2846.3
## <none>          1.7900 -2846.0
## + chas     1  0.004096 1.7859 -2845.2
## + crim     1  0.000772 1.7892 -2844.3
## - rad      1  0.143876 1.9338 -2808.9
## - age      1  0.163740 1.9537 -2803.8
## - dis      1  0.200402 1.9904 -2794.3
## - indus    1  0.234311 2.0243 -2785.8
##
## Step: AIC=-2895.78
## nox ~ dis + indus + age + rad + ptratio
##
##      Df Sum of Sq  RSS    AIC
## + medv     1  0.114544 1.5014 -2931.0
## + rm       1  0.040900 1.5751 -2906.8
## + lstat    1  0.034136 1.5818 -2904.6
## + black    1  0.010715 1.6053 -2897.1
## + tax      1  0.009732 1.6062 -2896.8
## <none>          1.6160 -2895.8
## + zn       1  0.002677 1.6133 -2894.6
## + crim     1  0.000814 1.6152 -2894.0
## + chas     1  0.000025 1.6159 -2893.8
## - dis      1  0.157309 1.7733 -2850.8
## - ptratio  1  0.173983 1.7900 -2846.0
## - age      1  0.179354 1.7953 -2844.5
## - rad      1  0.245005 1.8610 -2826.3
## - indus    1  0.300534 1.9165 -2811.5
##
## Step: AIC=-2930.98
## nox ~ dis + indus + age + rad + ptratio + medv
##
##      Df Sum of Sq  RSS    AIC
## + crim     1  0.012274 1.4892 -2933.1
## <none>          1.5014 -2931.0
## + chas     1  0.005168 1.4963 -2930.7
## + tax      1  0.002301 1.4991 -2929.8
## + lstat    1  0.001656 1.4998 -2929.5
## + black    1  0.001186 1.5003 -2929.4
## + zn       1  0.000079 1.5013 -2929.0
## + rm       1  0.000007 1.5014 -2929.0
## - medv     1  0.114544 1.6160 -2895.8
## - age      1  0.118964 1.6204 -2894.4
## - indus    1  0.177975 1.6794 -2876.3
## - dis      1  0.214794 1.7162 -2865.3

```

```
## - rad      1  0.237704 1.7391 -2858.6
## - ptratio  1  0.262429 1.7639 -2851.5
##
## Step:  AIC=-2933.13
## nox ~ dis + indus + age + rad + ptratio + medv + crim
##
##           Df Sum of Sq    RSS    AIC
## <none>                 1.4892 -2933.1
## + chas      1  0.004595 1.4846 -2932.7
## + tax       1  0.002185 1.4870 -2931.9
## + black     1  0.002122 1.4870 -2931.8
## + lstat     1  0.000978 1.4882 -2931.5
## + rm       1  0.000040 1.4891 -2931.2
## + zn       1  0.000016 1.4891 -2931.1
## - crim     1  0.012274 1.5014 -2931.0
## - age      1  0.118227 1.6074 -2896.5
## - medv     1  0.126004 1.6152 -2894.0
## - indus    1  0.168607 1.6578 -2880.9
## - dis      1  0.223460 1.7126 -2864.4
## - rad      1  0.228467 1.7176 -2862.9
## - ptratio  1  0.270558 1.7597 -2850.7
```

summary(MF)

```
##
## Call:
## lm(formula = nox ~ dis + indus + age + rad + ptratio + medv +
##      crim, data = Boston)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.122633 -0.035593 -0.004273  0.030938  0.182914
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  0.7750308  0.0349903  22.150 < 2e-16 ***
## dis         -0.0175329  0.0020282  -8.645 < 2e-16 ***
## indus        0.0044875  0.0005976   7.509 2.77e-13 ***
## age         0.0008656  0.0001377   6.288 7.04e-10 ***
## rad         0.0037478  0.0004288   8.741 < 2e-16 ***
## ptratio     -0.0132746  0.0013956  -9.512 < 2e-16 ***
## medv       -0.0022716  0.0003499  -6.491 2.06e-10 ***
## crim       -0.0007603  0.0003753  -2.026  0.0433 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.05468 on 498 degrees of freedom
## Multiple R-squared:  0.7804, Adjusted R-squared:  0.7773
## F-statistic: 252.8 on 7 and 498 DF, p-value: < 2.2e-16
```

El modelo final se modeliza a través de la siguiente ecuación

$$\text{nox} = 0.7750 - 0.01753 \text{dis} + 0.00449 \text{indus} + 0.00087 \text{age} + 0.00375 \text{rad} - 0.01327 \text{ptratio} - 0.00227 \text{medv} - 0.00076 \text{crim} + \varepsilon$$

Este modelo es bueno tiene un R^2 ajustado de 0.78 y además todas las variables son significativas, en base a los contrastes fundamentales (nos lo dicen los p-valores asociados), y también este modelo rinde mejor que el constante, también lo sabemos por el p-valor asociado al modelo. Teniendo en cuenta en rango intercuartílico y el la desviación típica de los errores, parece que puede tener buenas predicciones. Si bien es cierto que este es el mejor modelo, y además el AIC penaliza el uso de mayor cantidad de variables, el segundo mejor modelo difiere tan solo por 3 unidades de AIC. Al diferenciarse solo en la variable crim y no tener realmente esta, tanto peso en el modelo pues el p-valor asociado es muy cercano al nivel de significación, podríamos plantearnos tomar el segundo mejor modelo, el que no tiene a crim. Pues este segundo mejor modelo también sería más simple al tener una variable menos y mantendría métricas de rendimiento similares al obtenido como mejor modelo mediante step by step both.

9.7 Estudie la posible existencia de colinealidad en los predictores propuestos en el modelo Mf .

```
vif(lm(nox ~ dis + indus + age + rad + ptratio + medv + crim, data=Boston))
```

```
##      dis      indus      age      rad ptratio      medv      crim
## 3.080308 2.838750 2.535783 2.353934 1.541577 1.749296 1.759631
```

‘Dis’ es la que más colinealidad presenta pero en definitiva sería por un factor de aproximadamente un R^2 del 0.6. El resto de variables presentan un índice de colinealidad incluso menor. No parece haber ninguna variable que presente un nivel de colinealidad preocupante como no verificarse las hipótesis del modelo.

9.8 Comparación de las medidas de bondad de ajuste de M3 y Mf : R^2 , R^2 adj , error standard residual y Raíz del Error Cuadrático Medio de los residuos (RCME). Realice una tabla en que se comparen las medidas de bondad de ajuste de estos modelos. Comente los resultados obtenidos.

```
tabla_bondad <- data.frame(
  Modelo = c("M3", "Mf"),
  R2 = c(
    summary(M3)$r.squared,
    summary(MF)$r.squared
  ),
  R2_ajustado = c(
    summary(M3)$adj.r.squared,
    summary(MF)$adj.r.squared
  ),
  Error_std_residual = c(
    summary(M3)$sigma,
    summary(MF)$sigma
  ),
  RCME = c(
    sqrt(mean(residuals(M3)^2)),
    sqrt(mean(residuals(MF)^2))
  )
)

tabla_bondad
```


##	Modelo	R2	R2_ajustado	Error_std_residual	RCME
## 1	M3	0.7167775	0.7150849	0.06185253	0.06160757
## 2	Mf	0.7803909	0.7773040	0.05468346	0.05424945

En R2 y el ajustado es mas conveniente escoger el modelo MF, además mantiene la desviación típica de los residuos más baja así como el RCME. Por lo tanto no existe ningún incentivo para elegir M3 sobre el modelo MF a pesar de ser M3 un modelo más simple. Es importante mencionar que M3 no verifica las hipótesis de un modelo lineal, luego no sería válido y respecto a MF a pesar de no haberlo comprobado, será probable que tampoco las verifique. En este sentido si fuera un ejercicio real (suponiendo que MF tampoco verifica las condiciones para modelos lineales) no escogería ninguno.